NBA Player Analysis

Tyler Stoen

Data Overview

- Provided by Kaggle:

  2021-2022 NBA Player Statistics - Regular Season

- Raw data provided in CSV format
  - 30 features
  - 812 rows
    - Each row represents a player
    - Players traded this season have separate rows for each team the appeared on and a combined row

Reading in Data

- Raw CSV file converted to RDD
  - Raw data file included header - extract w/ RDD.first() & filter()
- Each row was parsed and mapped to a Player() class
- Resulted in RDD of player objects

```
val file = sc.textFile("C:/Users/Tyler/csc369/project/src/main/scala/2021-2022 NBA Player Stats - Regular.csv")
val header = file.first()
val colNames = header.split(";")
println(s"Column Names: ${colNames.mkString(", ")}")
val data = file.filter(row => row != header)
val reg_season_data = data.map{ line =>
  val stats = line.split(";")
  Player(
    id = stats(0),
    name = stats(1),
    position = stats(2),
    age = stats(3).toInt,
    team = stats(4),
    games_played = stats(5).toInt,
    games_started = stats(6).toInt,
    avg_min = stats(7).toDouble,
    avg_FGM = stats(8).toDouble,
    avg_FGA = stats(9).toDouble,
    pct_FG = stats(10).toDouble,
    avg_3ptM = stats(11).toDouble,
    avg_3ptA = stats(12).toDouble,
    pct_3pt = stats(13).toDouble,
    avg_2ptM = stats(14).toDouble,
    avg_2ptA = stats(15).toDouble,
    pct_2pt = stats(16).toDouble,
    pct_eFG = stats(17).toDouble,
    avg_FTM = stats(18).toDouble,
    avg_FTA = stats(19).toDouble,
    pct_FT = stats(20).toDouble,
    OREB_PG = stats(21).toDouble,
    DREB_PG = stats(22).toDouble,
    RPG = stats(23).toDouble,
    APG = stats(24).toDouble,
    STL_PG = stats(25).toDouble,
    BLK_PG = stats(26).toDouble,
    TOV_PG = stats(27).toDouble,
    FOULS_PG = stats(28).toDouble,
    PPG = stats(29).toDouble
  )
}
```

Hypothetical Situation

For the purposes of the project and analysing the given data, suppose my role is a member of the Lakers' front office. After a disappointing 2021-2022 season, I am looking to use the given data to determine how to improve the team for the upcoming season.

Key Goals

Researched online to find 2 main offseason goals following the 2021-2022 season:

1. Improve 3-pt shooting

2. Replace Russell Westbrook

3 Point Shooting - Team Rankings

Objective: Rank teams by three-point shooting efficiency.

Steps:

1. Aggregate total 3-point attempts and makes per team.
2. Compute 3PT% = (Total Makes / Total Attempts) * 100.
3. Sort teams in descending order by 3PT%.

```
// Calculate team-level 3PT stats
val team_3pt = reg_season_data
  .filter(team => team.team != "TOT")
  .map(player => (player.team, (player.games_played * player.avg_3ptA, player.games_played * player.avg_3ptM)))
  .reduceByKey((a, b) => (a._1 + b._1, a._2 + b._2)) // Aggregate attempts and makes per team
  .mapValues { case (totalAttempts, totalMakes) =>
    if (totalAttempts > 0) (totalMakes / totalAttempts) * 100 else 0.0 // Calculate 3PT%
  }

// Sort teams by 3PT% in descending order
val rankedTeams = team_3pt
  .sortBy({ case (_, pct3pt) => pct3pt }, ascending = false)
```

3 Point Shooting - Team Rankings

1, MIA, 38.19
2, ATL, 37.83
3, LAC, 37.04
4, CHI, 36.93
5, PHI, 36.77
6, GSW, 36.75
7, MIL, 36.56
8, PHO, 36.40
9, CHO, 36.39
10, BRK, 36.10
...
22, LAL, 34.64
23, POR, 34.53
24, SAC, 34.46
25, WAS, 34.44
26, IND, 34.43
27, NOP, 33.04
28, DET, 32.73
29, ORL, 32.68
30, OKC, 32.18

Finding 3pt Shooters

Objective: Identify top-performing shooters based on 3PT%.

Steps:

1. Filters
2. Sort players by 3PT% in descending order.
3. Select top 5 players.

Filter Criteria:

- avg_min > 20
- avg_3ptA >= 2
- games_played >= 40

```
val top_5_shooters = reg_season_data
  .filter(player =>
    (player.team == "TOT" || !tradedPlayers.contains(player.name)))
  .filter(player => (player.team != "LAL")
      && (player.avg_min > 20)
      && (player.avg_3ptA >= 2)
      && (player.games_played >= 40))
  .sortBy(player => player.pct_3pt * -1).take(5).foreach(println)
```

Suggestions - 3pt Shooters

1. Luke Kennard, SG, 25, 44.9%
2. Rui Hachimura, PF, 23, 44.7%
3. Isaiah Roby, PF, 23, 44.4%
4. Desmond Bane, SF, 23, 43.6%
5. Tyrese Maxey, PG, 21, 42.7%

Pictured above: Rui Hachimura

Russell Westbrook

Age: 33

Position: PG

Games Played/Started: 78

34.3 minutes/game

In short: Very important player

Replacing Westbrook

- Find candidates similar to Westbrook based on 2021-2022 regular season statistics
- Similarity measured using standardized Euclidean Distance
  - To account for different feature scales / outcome ranges

```
def standardizedEuclideanDistance(features1: Array[Double], features2: Array[Double], means: Array[Double], stdDevs: Array[Double]): Double = {
  Math.sqrt(features1.zip(features2).zip(means.zip(stdDevs)).map {
    case ((x1, x2), (mean, stdDev)) =>
      val z1 = if (stdDev == 0) 0.0 else (x1 - mean) / stdDev
      val z2 = if (stdDev == 0) 0.0 else (x2 - mean) / stdDev
      Math.pow(z1 - z2, 2)
  }.sum)
}
```

Initial Method

Compare all numerical features with Euclidean Distance

Restraint: Must be same position as Wesbrook

```
val distances = reg_season_data
  .filter(_.name != "Russell Westbrook")
  .filter(player => (player.position == "PG" || player.position == "PG-SG"))
  .map(player => (player, standardizedEuclideanDistance(numericalFeatures(player), westbrook_values, means, stdDevs)))

val topSimilarPlayers = distances
  .sortBy(_._2)
  .take(5)
```

Initial Method Results

```
Top Players Most Similar to Russell Westbrook by raw stats:

Player: Jrue Holiday, Distance: 4.2048828238828 58
Player: Dejounte Murray, Distance: 4.289978320812539
Player: De'Aaron Fox, Distance: 4.298037792990047
Player: Malcolm Brogdon, Distance: 4.310554260691211
Player: Cade Cunningham, Distance: 4.62989966000846
```

Problem:

- Recommends only players with high minutes due to including metrics directly impacted by the number of minutes played.
- Won’t help find underappreciated players

Revised Method

Solution: Scale all metrics directly impacted by playtime to “per 36 minutes”

- Ex. PPG ➔ points per 36 min

Same process as before.

```
def per36minFeatures(player: Player): Array[Double] = Array(
  player.age,
  player.avg_FGM / player.avg_min * 36,
  player.avg_FGA / player.avg_min * 36,
  player.pct_FG,
  player.avg_3ptM / player.avg_min * 36,
  player.avg_3ptA / player.avg_min * 36,
  player.pct_3pt,
  player.avg_2ptM / player.avg_min * 36,
  player.avg_2ptA / player.avg_min * 36,
  player.pct_2pt,
  player.pct_eFG,
  player.avg_FTM / player.avg_min * 36,
  player.avg_FTA / player.avg_min * 36,
  player.pct_FT,
  player.OREB_PG / player.avg_min * 36,
  player.DREB_PG / player.avg_min * 36,
  player.RPG / player.avg_min * 36,
  player.APG / player.avg_min * 36,
  player.STL_PG / player.avg_min * 36,
  player.BLK_PG / player.avg_min * 36,
  player.TOV_PG / player.avg_min * 36,
  player.FOULS_PG / player.avg_min * 36,
  player.PPG / player.avg_min * 36
)
```

Revised Results

```
Most Similar players to Russell Westbrook by AVG/36min stats:

Player: De'Aaron Fox, Distance: 5.5915656650940 96E-4
Player: Eric Bledsoe, Distance: 7.1150195706384 21E-4
Player: Malcolm Brogdon, Distance: 7.3173696874078 16E-4
Player: Dejounte Murray, Distance: 7.6789274889365 76E-4
Player: Brandon Williams, Distance: 7.7048261817395 74E-4
```

What I learned

- Applying Scala/Spark skills to new scenarios
- Debugging Scala/Spark code
- Similarity using Euclidean Distance
  - Compare different distance metrics in the future
- Domain Knowledge - NBA analytics

Challenges

- Finding data - initially wanted to make it more broad
- Deciding what to analyze
  - Lots of possibilities with this dataset
- Parsing datasets into RDDs efficiently
  - Tedious
  - Dataframes
- Edge cases
  - Players playing for multiple teams
  - Players with 0 or low minutes (sample size)

Thank You!